Engineering Guide

Architecture, Formats, Integration, Security, and Validation

Version 1.0

Last changed: August 12, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

The table of contents is refreshed during document finalization.

1. Product Scope and Invariants

Delphi App Translation Studio is a Windows-only, open-source localization workspace written in Delphi FireMonkey. It targets Delphi VCL and FMX applications compiled for Win32 and Win64. It is not a runtime translation service and does not require target machines to have Internet access.

The central invariant is separation: scan and catalog operations do not modify target source; provider access exists only in the Studio; recommended Component Integration writes only to the Studio export tree; advanced automatic integration remains explicit, previewed, transactional, backed up, and reversible; target applications consume compact offline JSON only.

- All user interface controls are persisted in DAT.Studio.MainForm.fmx and remain editable in the RAD Studio designer.
- No UI is constructed at runtime.
- Stable keys, not source text alone, identify translated properties.
- Original designer text remains the runtime fallback.
- API keys never enter project artifacts.
- One designer-persisted manager supervises the application; ordinary forms require no component.

2. Repository and Build Layout

Path	Responsibility
source\core	Catalog types, JSON persistence, project detection, workspace paths, runtime pack generation.
source\scan	VCL/FMX form text, Pascal resourcestrings, extraction rules, diagnostics, incremental merge.
source\validation	Catalog safety and completeness checks.
source\provider	Provider types/settings, Credential Manager, DeepL/Google HTTPS client.
source\runtime	Pack discovery/loading, preference, manager, VCL and FMX applicators.

Path	Responsibility
source\components	Framework-neutral manager core, VCL/FMX lifecycle adapters, and supplied language selectors.
source\design	VCL and FMX Tool Palette registration units.
packages\runtime / packages\design	Core and framework runtime BPLs plus Win32 IDE design packages.
source\integration	Non-mutating component kits plus advanced planning, resource/source changes, transactions, and reset.
source\studio	Designer-authored FMX UI and Studio self-localization.
source\schemas	Development and runtime JSON Schemas.
tools\tests	Foundation, runtime, integration, form-streaming, launch, and self-localization smoke tests.
docs / help / samples	Product documentation, help source, and safe fixtures.

2.1 Toolchain

```
call "C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat"
msbuild DelphiAppTranslationStudio.dproj /t:Build /p:Config=Debug /p:Platform=Win32
msbuild DelphiAppTranslationStudio.dproj /t:Build /p:Config=Release /p:Platform=Win64
```

The DPROJ identifies the main form as FMX, includes the standard project resource, uses source\provider in its unit search path, and routes EXE/DCU output to bin and dcu by platform/configuration.

3. System Architecture

Boundary	Input	Output
Detection and scan	.dproj/.dpr, text DFM/FMX, Pascal source	TProjectProfile and TProjectScanResult

Boundary	Input	Output
Catalog	Scan result plus existing development JSON	Merged TTranslationCatalog
Provider	Confirmed source strings plus in-memory credential	Machine-translated entry values
Validation	Development catalog	Errors, warnings, information
Pack builder	Valid catalog	Compact runtime JSON
Component integration	Project profile, scanner roots, packs	Non-mutating component kit under Studio export
Advanced integration	Project profile, packs, designated menu	Preview package and transactional change set
Runtime	Local JSON and per-user preference	Translated existing controls and locale settings

4. Core Data Model and JSON

DAT.Core.Types defines target framework/platform flags, locale metadata, translation status, catalog entries, and catalog ownership. Development JSON retains source text, translation, stable key, checksum, status, source location, component/property metadata, and locale data. Runtime JSON deliberately omits development-only detail.

4.1 Stable keys

- Form properties use form.component.property, for example frmMain.btnSave.Text.
- resourcestring entries use the Pascal unit and symbol.
- Keys remain stable when wording changes, allowing source-changed detection.
- Fallback source text remains in the compiled DFM/FMX or code.

4.2 Development catalog

```
{
  "schemaVersion": 5,
  "applicationId": "SampleApp",
  "sourceLanguage": "en-US",
```

```
  "locale": {"languageCode": "it-IT", "nativeLanguageName": "Italiano"},
  "entries": [{"key": "frmMain.btnSave.Text", "sourceText": "Save",
               "translatedText": "Salva", "status": "machineTranslated",
               "translationOrigin": "google",
               "runtimeApplication": "automatic"}]
}
```

4.3 Runtime pack

```
{
  "schemaVersion": 2,
  "applicationId": "SampleApp",
  "languageCode": "it-IT",
  "language": {"code": "it-IT", "nativeName": "Italiano", "direction": "ltr"},
  "sourceCatalogChecksum": "...",
  "strings": {"frmMain.btnSave.Text": "Salva"},
  "sources": {"frmMain.btnSave.Text": "Save"},
  "sourceStrings": {"Save": "Salva"},
  "sourceTemplates": {}
}
```

The schema files are source\schemas\development-project.schema.json and runtime-language-pack.schema.json. Changes require schema-version handling, round-trip fixtures, and compatibility notes.

5. Project Detection and Scanning

DAT.Core.ProjectDetection reads project metadata, resolves the DPR, detects VCL or FMX through project/form evidence, records Win32/Win64 support, and enumerates forms and Pascal sources. DAT.Scan.Project coordinates form and Pascal scanners while measuring elapsed milliseconds. Generated folders and nested directories that contain another DPROJ or DPR are excluded so a conversion utility is not mistaken for part of the selected application.

DAT.Scan.FormText parses text DFM/FMX without instantiating target forms, including Delphi multiline string expressions. DAT.Scan.PascalResources extracts resourcestring declarations plus supported runtime UI assignments and Format templates. It rejects SQL and HTML payloads that merely use Text properties. DAT.Scan.TextCodec preserves Delphi text encodings and escaped string syntax. DAT.Scan.Rules centralizes designated property decisions.

5.1 Incremental merge

1. Index the existing catalog by stable key.
2. Add unseen keys as needs-translation.
3. Preserve translation and status when source text is unchanged.
4. Preserve existing translation but mark source-changed when source text differs.
5. Mark catalog-only entries obsolete unless excluded.
6. Report new, changed, unchanged, and obsolete counts.

6. Studio UI and Workflow State

DAT.Studio.MainForm.fmx contains the complete orange-and-blue interface. The form persists WindowState=wsMaximized, uses client-aligned workflow cards, anchors resizable work areas to every relevant edge, and keeps the status card at the bottom. The workflow selection rectangle moves among Project, Scan, Translate, Validation, Export, Integration, and Provider Settings. Language and provider choices, Translate Automatically, key-management actions, and all other controls are persisted designer objects. DAT.Studio.MainForm.pas contains event and state logic only; it does not construct controls.

Each workflow TLabel explicitly persists HitTest=True. FireMonkey labels default to HitTest=False, which would otherwise pass mouse events through the visible label even when an OnClick event is assigned. The setting remains editable in the Object Inspector.

The form owns the project profile, scan result, catalog, validation result, integration change set, provider settings, and per-provider session-key strings. Destructors release owned objects. Catalog updates invalidate validation and export state. The designer-authored Integration method combo defaults to Component Integration; mutation controls are hidden in that mode and restored only for the explicitly selected advanced mode.

> **FMX validation requirement.** A successful compile is insufficient. Persisted FMX property-type errors surface only while streaming. StudioFormSmokeTests directly constructs the form, and launch tests verify the real title in every configuration.

6.1 Setup Wizard Architecture

DAT.Studio.SetupWizard is a separate, designer-authored FMX modal form opened by Start Setup Wizard. It is borderless, fixed-size, centered, and deliberately omits menu and system controls. Its nine hidden TabControl pages, including the Deployment Destinations page, left navigation rail, footer controls, labels, dialogs, lists, check boxes, memos, and layout geometry are all persisted in DAT.Studio.SetupWizard.fmx and remain editable in the Form Designer. A designer-authored modal backdrop dims and blocks the Studio while the Wizard is open so the two blue interfaces remain visually distinct.

The Wizard owns isolated project-profile, scan-result, catalog, provider-key, backup, and component-kit state. Steps already reached may be revisited; future steps remain disabled. Changing the project or target language invalidates downstream scan/catalog state. Before final processing, the target project is read only except for an explicitly saved provider credential outside the project. During final processing, navigation and closing are disabled.

Final processing requires confirmation that the target project is closed, creates a timestamped ZIP, translates eligible unresolved entries, saves the development catalog, validates, exports the runtime JSON pack, generates the component kit, deploys packs to existing outputs, and transactionally inserts

marked Search Path and PostBuildEvent properties inside Delphi's native Base compiler property group. Base inheritance covers Debug/Release and Win32/Win64. The Wizard never edits target Pascal, form, or DPR files and never writes RAD Studio's package registry.

> **Wizard regression requirement.** StudioFormSmokeTests must instantiate the Wizard and verify every primary designer component, including dlgOpenProject. The project-selection access violation of August 10, 2026 was caused by a declared TOpenDialog omitted from the FMX resource; the explicit presence assertion prevents that class of streaming omission from returning.

7. Provider Settings and Secret Storage

DAT.Provider.Settings persists only provider, DeepL plan, remember choice, timeout, and batch size in %LOCALAPPDATA%\DelphiAppTranslationStudio\provider-settings.json. Bounds are normalized to 5-300 seconds and 1-50 strings.

DAT.Provider.CredentialStore uses CredWriteW, CredReadW, CredDeleteW, and CredFree with CRED_TYPE_GENERIC and local-machine persistence. Credential targets are provider-specific and begin TMartinDub/DelphiAppTranslationStudio/. Secret bytes are cleared after conversion where practical.

Choice	Persistence	Replacement behavior
Remember checked	Windows Credential Manager	Writes/replaces selected provider credential and clears session copy.
Remember cleared	Process memory only	Deletes selected provider stored credential and retains entered key until shutdown.
Remove Key	None	Deletes stored and session copies for selected provider.

- The masked edit never displays a stored credential.
- The effective key resolves new field text, then session memory, then Credential Manager.
- No error message contains the key, request headers, or provider response body.
- Credential operations occur only when needed; the form can start without network access.

8. Provider HTTP Clients

DAT.Provider.Client uses Delphi THTTPClient. It validates a nonblank key, normalizes settings, builds provider-specific JSON, sends HTTPS POST requests, parses provider response arrays, verifies result counts, and returns translations in source order.

Provider	Endpoint	Authentication	Payload
DeepL API Free	https://api-free.deepl.com/v2/translate	Authorization: DeepL-Auth-Key	text array, source_lang, target_lang, context
DeepL API Pro	https://api.deepl.com/v2/translate	Authorization: DeepL-Auth-Key	text array, source_lang, target_lang, context
Google Basic v2	https://translation.googleapis.com/language/translate/v2	X-Goog-Api-Key	q array, source, target, format=text

8.1 Reliability and cancellation

- Batch size is capped at 50 and Google language tags are reduced to base language where appropriate.
- DeepL source codes are normalized to two-letter uppercase; targets retain supported regional forms.
- DeepL receives a batch context assembled from each entry's inferred UI role and semantic concept. DeepL documents context as unbilled supporting text.
- Google Basic v2 has no context or glossary request field. Context is used locally for terminology, memory matching, and ambiguity warnings but is not sent to Google.
- HTTP 429 and 5xx responses receive three bounded attempts with short backoff.
- Other non-2xx statuses fail immediately with provider, status, and corrective categories.
- A cancellation callback is checked between batches.
- Test Connection translates one harmless English phrase to Italian.

8.2 Official protocol references

- DeepL authentication: https://developers.deepl.com/docs/getting-started/auth
- DeepL quickstart: https://developers.deepl.com/docs/getting-started/quickstart

- Google Basic translate method: https://docs.cloud.google.com/translate/docs/reference/rest/v2/translate
- Google Cloud Translation authentication: https://docs.cloud.google.com/translate/docs/authentication
- Google API-key best practices: https://docs.cloud.google.com/docs/authentication/api-keys-best-practices

9. Automatic Provider Translation and Review

Translate Automatically is the canonical machine-translation path. It validates the selected catalog language, loads the selected provider settings and effective key, counts eligible unresolved entries, and shows the provider and count before any network request.

Every scan entry receives ContextKind, ContextDescription, SemanticConcept, and ContextConfidence metadata. Before a network request, the Studio reuses a Reviewed or Approved translation only when source text and semantic context match, then applies supported vetted UI terminology. The same authoritative terminology repairs unreviewed Google or DeepL drafts when a definitive application term is known, including commands, media playback, schedule terms, weekday abbreviations, and runtime uptime templates. Remaining strings are sent through DAT.Provider.Client in bounded batches. Reviewed, Approved, Edited, Excluded, and Obsolete entries are not overwritten.

DeepL receives contextual descriptions. Google Basic results are tagged provider-basic; short terms with unknown meaning receive a review note. DeepL contextual results are tagged contextual-provider. Terminology and translation-memory results retain their own provenance.

If no key is available, the workflow opens Provider Settings and gives a specific corrective message. HTTP or parsing failures leave the catalog entries unchanged for the failed operation and report a redacted summary in the status card.

CSV interchange and manual editing remain optional alternatives for translation-company collaboration or specialized review, not prerequisites for automatic translation.

The Studio may automatically reuse a Reviewed or Approved translation when both source text and semantic context match. It does not reuse approval state: the new entry remains Machine translated for review. Suggestions remain available for deliberate acceptance of other exact-source candidates.

- Mark Reviewed requires nonblank translated text.
- Approve requires the entry to have reached Reviewed first.
- Review All changes every nonblank active draft that is not already Reviewed, Approved, Excluded, or Obsolete after one count-bearing confirmation. Approve All changes only Reviewed entries after a second confirmation. Each bulk operation invalidates prior validation and saves the canonical JSON catalog once.

- Translation origin is independent of linguistic status.
- Inconsistent repeated terms and structural defects form the focused exception queue; a valid machine-translated draft is not itself an error.
- Structural validity, linguistic status, automatic runtime coverage, and manual wiring readiness are separate measures.
- A Pascal resourcestring remains manual wiring until the developer confirms the generated TranslateText call site.

10. Validation and Runtime Pack Export

DAT.Validation.Catalog validates metadata, missing target text, duplicates, changed source, Delphi indexed and sequential Format arguments, accelerators, inconsistent repeated-source terminology within the same semantic context, ambiguous context, and status conditions. Export is blocked by errors. Common runtime placeholders such as -- are informational when intentionally unchanged. The UI explains severity and maps a double-clicked entry issue back to the Translate list. Manual resourcestring wiring is reported separately from structural errors. DAT.Core.RuntimePack serializes only runtime-required metadata and strings and records a source-catalog checksum.

Locale data is retained in the runtime pack so DAT.Runtime.Manager can expose a pack-specific TFormatSettings without globally mutating the developer's source code.

11. Runtime Engine

DAT.Runtime.LanguagePack loads and discovers JSON packs, checks application identity, rejects empty/invalid packs, canonicalizes native language names, de-duplicates exact locale codes, and suppresses a generic locale when a regional variant exists. Runtime schema 2 retains keyed strings and templates while adding source text, source-string, and source-template indexes; schema 1 packs remain readable. DAT.Runtime.Preference reads/writes the selected locale. DAT.Runtime.Manager owns the active pack, discovery, preference, translation lookup, and locale format settings. The source language loads its generated JSON pack when present and falls back to designer text only for older deployments without that pack.

DAT.Runtime.VCL and DAT.Runtime.FMX traverse existing component trees and supported collection properties. They apply values by stable key to already designer-created controls. The FMX adapter also translates anonymous runtime-created components, grid cell text, and supported browser-generated text through the source indexes. Missing keys retain source text. Accepted JSON layout rules may adjust only the recorded safe properties for the active language; the adapters do not create controls or rewrite Delphi form source.

11.1 Component manager core

TDATCustomLanguageManager owns TTranslationRuntime, immutable post-initialization configuration, stable form-identity mappings, generation tracking, deterministic removal, main-thread and reentrancy guards, exclusion policy, diagnostics, and lifecycle events. SelectLanguage advances the generation, applies the active pack to open forms, saves the preference, and raises additive notifications.

TDATFMXLanguageManager subscribes additively to TFormBeforeShownMessage and TFormReleasedMessage. It translates after streaming but before OnShow/first paint and never replaces a global handler. Its designer-visible AutoRefreshDynamicText and DynamicRefreshInterval properties reapply the active pack to visible forms so timers and application code cannot permanently overwrite translated status text, menu captions, grid headers, or uptime displays. TDATVCLLanguageManager owns a private TApplicationEvents, discovers visible forms on throttled idle, and inspects modal forms before display. VCL has no public additive before-show notification for every dynamic modeless form; such a form can paint once in the source language unless the application calls ApplyToForm before Show.

PreserveControlState protects writable edit/memo data, focus, selection ranges, and list/combo ItemIndex. Collection replacement temporarily suppresses and then restores OnChange. Read-only instructional memo content remains translatable.

11.2 Required language-selection UI

TDATVCLLanguageComboBox and TDATFMXLanguageComboBox are the supplied designer-owned language selectors. Their typed LanguageManager property streams in DFM/FMX resources. At runtime AutoPopulate obtains validated descriptors, ShowLanguageCode controls display formatting, and selection calls the manager while preserving the inherited OnChange notification. RefreshLanguages supports packs deployed after startup. A connected Language menu is permitted instead, but the localized application must expose some visible language-selection UI.

11.3 Deployment paths

- Packs: <ExecutableDirectory>\Localization\Languages\<locale>.json.
- Preference: %LOCALAPPDATA%\<ApplicationId>\language.ini.
- Studio preference: %LOCALAPPDATA%\DelphiAppTranslationStudio\language.ini.
- Developer catalogs remain in the target source tree under Localization\Development.

12. Component Packages and Integration Modes

The package graph contains DATLanguageManagerCoreRuntime, framework-specific VCL/FMX runtime packages, and separate VCL/FMX design packages. Each design package contains its DAT

runtime/component units directly and therefore imports no custom DAT runtime BPL; this eliminates the IDE loader failure caused by missing dependent modules. Design packages register only their applicable manager and selector on DAT Localization. Runtime packages build for Win32 and Win64; RAD Studio consumes Win32 design BPLs.

The Studio's Show Design BPL action selects the matching, verified, self-contained package under bin\packages\Win32\Release. The developer installs that stable file with RAD Studio's Component > Install Packages > Add command. The Studio does not copy BPLs to public Delphi folders, write Known Packages registry values, close/restart RAD Studio, or expose automatic package installation. Delphi therefore remains the sole owner of design-package registration. Generated per-project kits contain no BPLs and are regenerated from a clean output directory, so they can be replaced without invalidating Delphi's registered package path.

DAT.Integration.ComponentPackage implements the recommended non-mutating mode. It emits applicable runtime/component units, validated translated packs, an English source pack, component-integration.json, scanner form roots, README instructions, and a deployment script exclusively below export\component-integration. SHA-256 fixture tests prove target DPROJ and DFM/FMX hashes remain unchanged.

The advanced fallback retains DAT.Integration.Plan, DAT.Integration.Package, DAT.Integration.Engine, MenuResource, DelphiSource, Transaction, BuildDeploy, and Reset. It generates the application unit and exact changes in memory, requires explicit authorization, verifies a pre-change backup, writes atomically, rolls back failures, and supports restore/complete reset.

12.1 Component startup contract

- The primary form streams one manager through ordinary Delphi designer mechanics.
- Loaded initializes the runtime, loads the preferred/source language, and applies the owner.
- FMX before-show messages cover later normal, inherited, and popup forms.
- VCL idle/modal discovery covers ordinary forms; strict modeless pre-display callers use ApplyToForm.
- SelectLanguage applies the pack to currently open forms immediately and persists the locale.
- The generated en-US pack makes switching back to source text deterministic.
- Manager and lifecycle subscriptions are released deterministically.

Original code preservation. Recommended Component Integration performs zero automatic writes to the target. The developer's normal act of placing the manager causes Delphi to persist one component field/resource and applicable uses unit. Original DFM/FMX text remains the source-language fallback.

13. Self-Localization

DAT.Studio.Translation resolves the project/deployed pack directory, initializes a Studio runtime before form creation, applies the active pack in FormCreate, and maps persisted language-menu names to locale codes. Package streaming fixtures run in isolated bin\tests\packages directories so their local JSON cannot shadow the Studio's project/deployed packs.

14. Error Handling and Diagnostics

- Project, scan, catalog, validation, export, integration, credential, and provider errors are caught at Studio event boundaries and summarized in the status card.
- Provider exceptions carry an optional HTTP status but not response bodies.
- Form scanners return file/line diagnostics without instantiating target UI.
- Integration previews show a complete line-numbered exact diff for every affected file before mutation.
- Transaction errors trigger rollback and retain backup evidence.
- Provider keys must never be added to troubleshooting logs.

15. Test Strategy and Verified Matrix

Test	Coverage
FoundationSmokeTests	Detection, scan-to-catalog, schema/provenance round-trip, provider contracts, review/approval, validation, runtime pack, preference, advanced integration, component-kit contents, and SHA-256 non-mutation proof.
Language manager suites	Core guards/generations, FMX before-show lifecycle, VCL discovery/modal boundary, stable identities, instant switching, state preservation, and deterministic cleanup on Win32/Win64.
Package/selector suites	Debug and Release runtime/design packages, DFM/FMX streaming, typed manager references, pack discovery, and selector language propagation on Win32/Win64.
VCLRuntimeSmokeTests	VCL controls, menu items, locale, generated unit.

Test	Coverage
FMXRuntimeSmokeTests	All representative FMX form properties, menu items, locale, generated unit.
StudioFormSmokeTests	Direct FMX stream/create, maximized client-aligned pages, provider-only automatic-translation controls, exact-review controls, linguistic action wiring, and Provider Settings activation.
RunRuntimeSmokeTests.ps1	Both compilers, scanner/catalog/provider contracts, disposable integrated VCL/FMX builds, deployed Italian pack, and required Italian launch title.
RunStudioLaunchSmokeTests.ps1	Debug/Release Win32/Win64 real main-window title.
RunStudioSelfLocalizationSmokeTest.ps1	Italian Studio title in all four configurations with state restoration.

On August 9, 2026, the complete release harness passed uninterrupted. Debug and Release component packages, Win32/Win64 manager suites, scanner/catalog/provider contracts, component non-mutation, runtime and advanced integration all passed. The Studio built in Debug and Release for both architectures, streamed its FMX form directly, launched normally, and self-localized to Italian in all four configurations. Disposable real-application pilots also built and opened translated first forms on Win32/Win64 for Website Analytics (FMX) and Courier Herald Reader (VCL).

15.1 Optional live provider testing

Automated fixtures must not contain live keys. Live provider acceptance uses an owner-supplied restricted key through Provider Settings, confirms Test Connection, translates a small disposable catalog, checks Machine translated status and provider provenance, then removes or rotates the test key. Provider availability is external state and cannot be made deterministic, but live provider acceptance is required before a public release that claims current Google or DeepL compatibility.

16. Security Review

Threat	Control
Key committed to Git	No key field in settings JSON/catalogs; Credential Manager or memory only; release secret scan.
Key leaked through URL/log	Authentication headers, redacted errors, no body/header logging.
Wrong target changed	Recommended component kit has no target-write path; advanced mode uses profile display, preview, and explicit Apply.
Unrecoverable source mutation	Verified G-drive project backup by workflow policy plus transaction backup/manifest/restore.
Installed-folder write failure	Per-user language preference; packs are read-only deployment assets.
Machine mistranslation shipped	Machine-translated status, validation, required human review.
Pack for wrong application	applicationId validation in pack discovery/loading.

Windows Credential Manager follows Microsoft guidance for application credentials (CredWrite/CredRead). It protects persistence under the signed-in Windows context; it does not make a key safe from malware running as that user. Provider-side restrictions, quotas, rotation, and least privilege remain necessary.

17. Release and Contribution Workflow

1. Read global and repository directives.
2. Confirm the exact requested scope and make a pre-change backup when changes are approved.
3. Keep UI edits in the FMX designer resource and source event logic separate.
4. Add deterministic tests without credentials.
5. Run Win32/Win64 Debug/Release builds elevated.
6. Run runtime, form-streaming, launch, and self-localization suites.
7. Update phase notes, help, User Guide, Engineering Guide, real TOCs, PDFs, and visual QA.
8. Check for a stale zero-byte .git\index.lock with no Git process.

9. Review status, stage only intended files, commit clearly, and push every configured remote.

18. Known Boundaries and Future-Compatible Design

- Only Windows Win32/Win64 Delphi applications are supported.
- Google Advanced v3, OAuth/service-account workflows, and other providers are not implemented.
- Provider operations are explicit Studio actions; target runtime remains offline.
- No automatic control resizing/reflow is performed.
- Binary DFM conversion is outside the scanner.
- Automatic runtime application covers scanned designer properties. Application-specific strings still require the documented TranslateText wiring.
- VCL dynamic modeless forms may paint once in source language before idle discovery; call ApplyToForm before Show when a no-flicker first display is mandatory.
- Provider account terms and language support are external and must be rechecked for each release.

19. Documentation Generation

The editable guides are generated from actual source and engineering notes with python-docx. Each DOCX contains a real TOC field, a dedicated TOC section, and a new-page content section. LibreOffice performs the approved headless PDF conversion with an isolated user profile. Microsoft Word is not used. Every final PDF is rendered to page images and inspected before release.

20. Source References

- Repository source, forms, project metadata, schemas, and smoke tests as of August 10, 2026.
- Microsoft credential handling: https://learn.microsoft.com/en-us/windows/win32/secbp/handling-passwords
- Windows CREDENTIAL structure: https://learn.microsoft.com/en-us/windows/win32/api/wincred/ns-wincred-credentialw
- DeepL developer documentation: https://developers.deepl.com/docs/getting-started/auth
- Google Cloud Translation documentation: https://docs.cloud.google.com/translate/docs/authentication
- Google API-key guidance: https://docs.cloud.google.com/docs/authentication/api-keys-best-practices